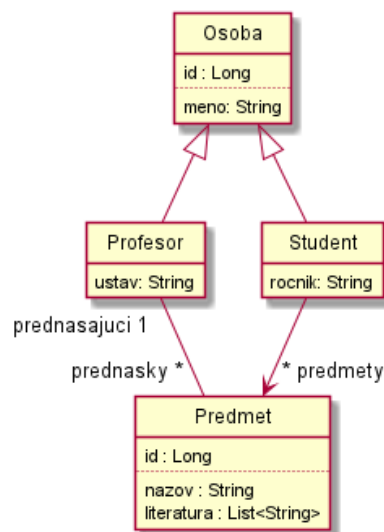
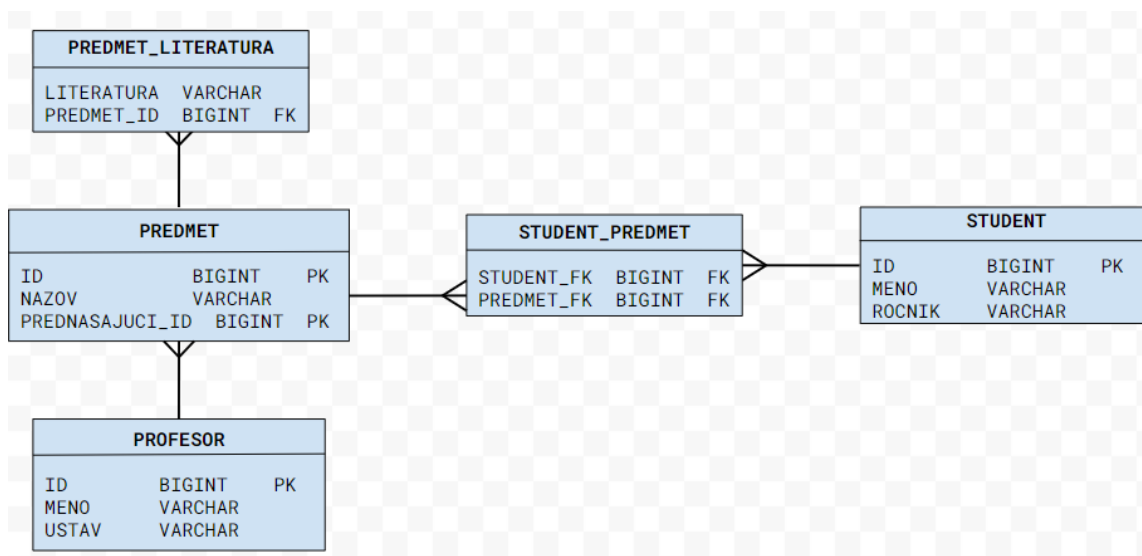


Adresár **dbanot** je maven projekt pre vývoj databázovej aplikácie. V balíku **vsa** sú vytvorené entitné triedy podľa UML diagramu tried, zatiaľ bez JPA anotácií:



Úloha: Triedy v balíku **vsa** **doplňte o JPA anotácie** tak, aby z nich vygenerovaná databáza mala štruktúru znázornenú na relačnom diagrame:



Dôležité:

- Názvy tabuliek a stĺpcov musia byť také ako je uvedené na diagrame. Všimnite si dobre predovšetkým názvy cudzích kľúčov. Poradie stĺpcov vo vygenerovanej databáze však nemusí byť presne také ako je na diagrame.
- Stĺpec NAZOV v tabuľke PREDMET má byť unikátny.
- Stĺpec ROCNIK v tabuľke STUDENT je povinný (nie je nullable).
- Do tried v balíku **vsa** stačí doplniť len JPA anotácie. Žiadne ďalšie modifikácie týchto tried ani ďalšie nové metódy či triedy nie sú potrebné.

Poznámky:

Projekt už obsahuje aj súbor **persistence.xml** s konfiguráciou pre pripojenie k databáze postgresql a tiež jednoduchý program, trieda **NewMain.java**, ktorý môžete použiť, ak si chcete vygenerovať databázu pre kontrolu vášho riešenia (nie je to však povinné).

Pokyny pre odovzdanie

Odovzdajte balík **vsa** zozipovaný ako súbor **vsa.zip**

Žiadam vás, na zipovanie použite len formát **zip** (nie rar či iné archívne formáty). Zipujte len samotný balík so zdrojovými súborami, nie celý projekt! Je to dôležité pre včasné vyhodnotenie vašich riešení.

Na obrázku je znázornený diagram databázovej tabuľky s informáciami o platoch zamestnancov. Ide o tzv. historizovanú tabuľku, ktorá uchováva celú históriu zmien (platu). Preto má zložený kľúč pozostávajúci z mena zamestnanca a dátumu zmeny platu.

ZAMESTNANEC		
MENO	VARCHAR	PK
DATUM_OD	DATE	PK
PLAT	DOUBLE	

Adresár **dbapp** obsahuje maven projekt pre vývoj aplikácie pracujúcej s touto tabuľkou.

V balíku **dbapp** sa nachádza súbor **Platy.java** s deklaráciami dvoch metód:

```
static void updateSalary( EntityManager em, String name, double salary )
```

```
static double salaryOn( EntityManager em, String name, LocalDate date )  
    throws Exception
```

Metóda **updateSalary** vytvorí v tabuľke ZAMESTNANEC nový záznam o plate zamestnanca:

Meno zamestnanca a jeho nový plat dostane ako argumenty.

Do stĺpca DATUM_OD vloží aktuálny dátum.

Upozornenie: metóda nesmie vyhodiť výnimku pri pokuse zadať v ten istý deň plat viac krát. V takom prípade ostane platná prvá zmena.

Metóda **salaryOn** vráti plat zamestnanca k danému dňu. Teda nájde v tabuľke záznam o plate zamestnanca s dátumom zmeny (DATUM_OD) ktorý je rovný dátumu v argumente (date) alebo je najbližší starší od neho (pre ním).

Úloha: Vytvorte entitnú triedu pre prácu s tabuľkou ZAMESTNANEC a implementujte hore uvedené metódy. Entitnú triedu a príp. ďalšie triedy potrebné pre prácu s databázou **vytvárajte** tiež **priamo v balíku dbapp**.

Informácia k práci s triedou LocalDate:

- Triedu **LocalDate** mapuje JPA automaticky na databázový typ **DATE**.
- Pre vytvorenie aktuálneho dátumu použite statickú metódu **LocalDate.now()**
- Pre vytvorenie konkrétneho dátumu má trieda statickú metódu **LocalDate.of(...)**
- Pre porovnávanie dátumov má metódy **isEqual(..)**, **isAfter(..)**, **isBefore(..)**

Poznámka: Projekt už obsahuje aj súbor **persistence.xml** s konfiguráciou pre pripojenie k databáze postgresql. Trieda **Platy** obsahuje už program, metódu **main**, ktorú môžete použiť, ak si chcete vygenerovať databázu a otestovať vašu implementáciu (nie je to však povinné).

Pokyny pre odovzdanie

Odovzdajte balík **dbapp** zozipovaný ako súbor **dbapp.zip**

V adresári **restapp2t** nájdete maven projekt pre vývoj webovej aplikácie s REST servisom.

Projekt obsahuje:

- súbor **pom.xml** so všetkými závislosťami potrebnými pre vývoj servisu. Nie je ho potrebné dopĺňať.
- balík **rest**. **V tomto balíku budete vytvárať všetky triedy** (triedu implementujúcu servis aj dátovú/é triedu)

V balíku rest sa už nachádza konfiguračná trieda REST aplikácie JAXRSConfiguration. Netreba ju meniť.

Špecifikácia servisu

Servis bude sprístupňovať informácie skúškach. Informácie o skúške z konkrétneho predmetu predstavujú resource, ktorého stav reprezentovaný vo formáte XML má štruktúru:

```
<?xml version="1.0" encoding="UTF-8"?>
<skuska>
  <predmet>VSA</predmet>
  <den>utorok</den>
  <student>Jozef Mrkvicka</student>
  <student>Janko Hrasko</student>
</skuska>
```

kde:

- element **predmet** obsahuje identifikátor predmetu a je zároveň **jednoznačným identifikátorom resoursu skúšky** (t.j. pre jeden predmet môže existovať len jedna skúška). Predmet musí byť zadaný.
- element **den** udáva **deň** konania skúšky.
- elementy **student** obsahujú mená študentov prihlásených na skúšku.

Poznámka. Uvedený dokument je ukážka. XML dokumenty, s ktorými servis pracuje budú mať elementy s rovnakými názvami a štruktúrou, odlišovať sa budú len textovým obsahom a počtom elementov. Prihlásených študentov môže byť viac ale nemusí byť žiaden. Poradie elementov nie je dôležité.)

Inicializácia servisu

Servis inicializujte tak, že po štarte (deploymente) bude vytvorený jediný resource, a to pre skúšku z predmetu **VSA**. Deň konania skúšky bude **utorok** a prihlásení nebudú žiadni študenti.

Informácie o skúškach budú udržiavané len v pamäti servisu. Vytvorte si pre ne vhodné dátové štruktúry a použite anotáciu **@javax.inject.Singleton**. Nezabudnite tiež na anotáciu **@XmlRootElement**.

Koreňový resource nazvite **skuska**.

Informácie o konkrétnej skúške sú prístupné ako subresource s URI **skuska/{predmet}**, kde reťazec **predmet** je identifikátor predmetu.

Pre koreňový resource skuska implementujete metódy pre požiadavky:

POST, ktorá slúži pre vytvorenie nového resoursu s informáciami o skúške.

- akceptuje XML dokument (MIME: APPLICATION_XML) podľa horeuvedenej špecifikácie.
- vracia reťazec (MIME: TEXT_PLAIN) s identifikátorom predmetu (prevzatým z elementu predmet).

Pozor. Metóda musí najprv preveriť, či resource pre skúšku z daného predmetu už neexistuje. Ak už existuje, neurobí nič a vráti reťazec **duplicita**.

GET pre TEXT_PLAIN: Vráti reťazec obsahujúci identifikátory všetkých skúšok (predmetov) aktuálne v systéme oddelené medzerou.

Požiadavka **GET** tiež umožňuje zistiť skúšky, na ktoré je prihlásený konkrétny študent. Meno študenta musí byť zadané ako parameter požiadavky **student** (`@QueryParam("student")`). Ak je tento parameter zadaný, metóda vyhladá a vráti len identifikátory skúšok, na ktoré je prihlásený daný študent. Ak študent nie je prihlásený na žiadnu skúšku, vráti text: **ziadne predmety**.

Pre resource `skuska/{predmet}` implementujte nasledujúce metódy:

GET pre MIME TEXT_PLAIN: vráti počet študentov prihlásených na skúšku z daného predmetu. Ak resource neexistuje, malo by sa vrátiť 0 prípadne HTTP 204 alebo HTTP 404 .

GET pre MIME APPLICATION_XML: vráti všetky informácie o skúške z daného predmetu ako XML dokument s horeuvedenou štruktúrou. Ak resource pre skúšku z daného predmetu neexistuje, malo by sa vrátiť HTTP 204 alebo 404 (*Pomôcka, implementacia vráti jednoducho null*)

POST slúži na prihlásenie študenta na skúšku.

- Očakáva reťazec s menom študenta, ktorý sa chce prihásiť (MIME: TEXT_PLAIN). Ak je študent s daným menom ešte nie je prihlásený, pridá ho medzi prihlásených študentov. Ak je už študent prihlásený, nerobí nič."
- Metóda vracia textovú spávu (MIME: TEXT_PLAIN), ktorá obsahuje **deň skúšky**. Ak už študent bol prihlásený, správa obsahuje okrem dňa aj reťazec **duplicita**. (napr. "utorok duplicita")
Ak predmet neexistuje, vráti text: **predmet neexistuje**.

Pokyny pre odovzdanie

Odovzdajte balík **rest** zozipovaný ako súbor **rest.zip**